

Coding Strategies for Multi-Agent Mathematical Discovery Systems: A Comprehensive Technical Guide

Building a 65-agent autonomous mathematical discovery engine requires a carefully phased evolutionary approach that transforms inert hardware infrastructure into a self-improving research system capable of novel mathematical discovery. This analysis synthesizes project-specific documentation with industry best practices to provide actionable implementation guidance across all seven development phases (0-6), revealing that the most effective multi-agent systems follow a strict evolutionary trajectory where **infrastructure precedes intelligence, verification precedes expansion, and governance precedes autonomy**.

The project's architectural philosophy—"build the city before the citizens arrive"—aligns with industry consensus that 80% of effort should focus on foundational infrastructure rather than agent capabilities. (Phase 0 Coding Strategy: Archi... CrewAI) Hardware constraints (Ryzen 7 8700F, RTX 4060 with 8GB VRAM, 32GB RAM) necessitate innovative serialization protocols, (Phase 0: Infrastructural Agents ...) while the six-gap framework (Search, Memory, Assumption, Conflict, Reliability, Evolution) provides a systematic roadmap for addressing the fundamental brittleness of flat multi-agent architectures.

Phase 0 establishes the immutable laws of the digital society

Phase 0 is not merely preparatory infrastructure but the construction of the system's **digital ontology and legislative framework**. Before any problem-solving agent is instantiated, the "physics" of agent interaction—how they communicate, remember, and discover one another—must be codified. Failure to execute Phase 0 with extreme rigor results in a "Tower of Babel" scenario where highly capable agents exist in isolation, unable to transmit the semantic nuance of mathematical concepts. (Phase 0 is not merely a preparat... build order Multi-Agent Mathe...)

The **hardware calibration strategy** addresses the critical 8GB VRAM bottleneck through the "One-Model-At-A-Time" rule. (Phase 0: Infrastructural Agents ...) (phases 0-6 for the Autonomous ...) A single quantized 7B parameter LLM requires approximately 5-6GB VRAM, making concurrent cognitive agents physically impossible. The implementation mandates a **Stateful Serialization Protocol** where agents act as lightweight prompts routed to a central LLM backend service, with agent weights swapped in and out as needed. (phases 0-6 for the Autonomous ...) The Agent Management System (AMS) must reject new instantiation requests when memory usage exceeds 90%.

The **semantic substrate** adopts OMDoc/OpenMath standards as the mandatory exchange format, implementing a three-layer schema: Object Level (encodes formulae like $x^2 + y$), Statement Level (encodes definitions, theorems, and proofs as distinct logical entities), and Theory Level (encodes modular mathematical theories). (Phase 0 is not merely a preparat... build order Multi-Agent Mathe...) This eliminates natural language ambiguity by conveying semantic content rather than presentation. (Phase 0 Coding Strategy: Archi... phases 0-6 for the Autonomous ...) Communication follows **FIPA-ACL protocols** based on Speech Act Theory, where every message constitutes a formal, binding action (build order Multi-Agent Mathe...) with required fields including (performative), (sender),

`receiver`, `content` (OMDoc payload), `ontology`, and mandatory `conversation-id` for tracking thread-specific states across asynchronous interactions. `Phase 0 Coding Strategy: Archi...`

The infrastructural triumvirate consists of three agents: the **Agent Management System** (AMS) as "City Hall" managing agent lifecycles and enforcing hardware constraints; the **Directory Facilitator** (DF) as "Yellow Pages" enabling dynamic capability discovery; and the **Agent Communication Channel** (ACC) as "Postmaster" guaranteeing message routing and queuing for inactive agents. `Phase 0 Coding Strategy: Archi...`

`build order Multi-Agent Mathe...` Memory architecture implements both a **Blackboard** (shared workspace with Publish-Subscribe semantics) and **Vector Database** (long-term archive using lightweight stores like Milvus or ChromaDB in persistent mode). `Phase 0: Infrastructural Agents ...`

Industry best practices validate this approach. The Linux Foundation's **Agent Communication Protocol (ACP)** uses RESTful architecture supporting synchronous/asynchronous communication `Agentcommunicationprotocol` with MIME-type-based content identification. Google's **Agent-to-Agent Protocol (A2A)** enables discovery through "Agent Cards"—self-descriptions outlining capabilities and accepted request types. `Auth0` Anthropic's **Model Context Protocol (MCP)** connects agents to tools and data sources using JSON-RPC. `Auth0` The relationship is clear: "ACP connects agents to agents; MCP connects agents to their tools and knowledge." `IBM`

Phase 0 Definition of Done:

- Core civic infrastructure (AMS, DF, ACC) deployed and actively monitoring resources
- FIPA-ACL and OMDoc protocols governing all interactions
- Blackboard and Vector Database operational
- System is "computationally alive but mathematically inert" `Phase 0 is not merely a preparat...`

Phase 1 constructs the hallucination-resistant cognitive chassis

Phase 1 marks the transition from static infrastructure to the system's first dynamic "heartbeat." The primary objective is constructing a reliable, **hallucination-resistant reasoning core**—not achieving broad mathematical capability. `Phase 1 is the construction of th...` `build order Multi-Agent Mathe...` The architectural mission is to "Shackle Neural Creativity to Symbolic Rigor," meaning every creative step generated by any component must be validated by a deterministic symbolic system before acceptance as truth. `Phase 1 Coding Strategy: Archi...`

The **Orchestrator-Prover-Verifier loop** establishes the core operational pattern. The Problem Analysis Team ("Gatekeepers") sanitizes chaotic natural language inputs into structured mathematical objects, `Phase 1 is the construction of th...` with a Syntax Parser translating raw text into OMDoc expression trees and a Structure Recognizer classifying problems as Computation, Proof, or Optimization. The Main Orchestrator implements **Hierarchical Task Network (HTN) logic** to break complex requests into sub-goals while maintaining a hard-coded "**Non-Intervention**" Directive—strictly forbidden from performing calculations, only delegating. `Phase 1 Coding Strategy: Archi...` `build order Multi-Agent Mathe...`

The **Pilot Solver** wraps SymPy as a "tracer bullet" to verify the end-to-end pipeline, while the **Verification Core** implements the Ax-Prover pattern with two-layer defense: a Logic Checker scanning for "illegal moves" (dividing by zero, invalid domain operations) and a Formal Verifier wrapper compiling candidate proofs in Lean 4, Coq, or Isabelle. (build order Multi-Agent Mathe...) No result releases without verification stamp.

(Phase 1 Coding Strategy: Archi...)

Industry research strongly supports this pattern. DeepMind's **AlphaProof** achieved IMO silver medal performance by combining a fine-tuned Gemini model with AlphaZero reinforcement learning and Lean for verifiable proofs. **AlphaGeometry** uses neuro-symbolic architecture—a neural language model predicting useful geometric constructs while a symbolic engine provides rigorous logical reasoning—solving 83% of historical IMO geometry problems. The **Thinker-Judge-Executor (MACM)** pattern, where proposals are validated before computation and rejected proposals trigger refinement loops, achieves 10%+ accuracy improvement on mathematical benchmarks. (Architectural Patterns for Robu...)

The neural-symbolic verification loop proceeds as follows: Verification Core intercepts candidate proof → Formal Verifier attempts compilation → SUCCESS: proof compiles, STATUS: VERIFIED, Orchestrator releases answer → FAILURE: ERROR FLAG raised, Orchestrator retries with different strategy.

(Phase 1 is the construction of th...)

Phase 1 Definition of Done:

- Functional Orchestrator-Prover-Verifier loop passing end-to-end tests
- System "architecturally complete, mathematically limited" to SymPy capabilities

(Phase 1 is the construction of th...)

- Critical architectural separation: Planning from Execution, Intuition from Verification

(Phase 1 Coding Strategy: Archi...)

Phase 2 builds the specialized domain workforce through systematic decomposition

Phase 2 executes **Domain Decomposition**—systematically fragmenting the monolithic "Math Agent" concept into a granular hierarchy of expert agents. The strategic objective prevents the "Jack of all trades, master of none" failure mode by ensuring specific mathematical operations execute through agents wrapping specific, optimized algorithms. (build order Multi-Agent Mathe...) The non-negotiable principle: algorithmic wrappers over generic LLM prediction, with LLM reasoning reserved for orchestration and strategy, never computation.

(Phase 2 Coding Strategy for Ve...)

The **three-tier hierarchy** structures as follows: Tier 1 (Main Orchestrator from Phase 1), Tier 2 (Domain Supervisors as strategic routers), and Tier 3 (Specialist Agents as computational workhorses).

(Phase 2 Coding Strategy for Ve...) Each domain implements specialized teams:

Algebra Team: Supervisor routes based on prerequisites (factoring before roots), with specialists for arbitrary-precision arithmetic (GMP or mpmath—standard floating-point FORBIDDEN), polynomial manipulation

(Gröbner Bases via Buchberger's algorithm), and number theory (Miller-Rabin primality, Pollard's rho factorization). (Phase 2 Coding Strategy for Ve... (build order Multi-Agent Mathe...

Calculus Team: Supervisor implements critical decision logic routing symbolic versus numerical approaches. The Integration Specialist uses a **DUAL-ENGINE approach** as the most hallucination-prone operation—Symbolic Engine with Risch Algorithm for closed-form antiderivatives, Numerical Engine with adaptive Simpson's quadrature as fallback. (build order Multi-Agent Mathe...) Additional specialists handle differentiation (gradients, Jacobians, Hessians), differential equations (boundary value recognition, Runge-Kutta selection), and series expansions. (Phase 2 Coding Strategy for Ve...

Linear Algebra Team: Supervisor analyzes matrix properties (sparse/dense, symmetric, positive-definite) to select optimal algorithms. Specialists handle matrix operations, decompositions (**SVD, QR, LU, Cholesky**), and vector space analysis (basis, rank, nullspace, eigenvalues). (Phase 2 Coding Strategy for Ve... (build order Multi-Agent Mathe...

Probability and Statistics Team: Supervisor enforces **philosophical separation** routing Bayesian versus Frequentist approaches based on request semantics. Bayesian Inference Engine implements MCMC methods while Frequentist Agent handles classical hypothesis testing. (Phase 2 Coding Strategy for Ve... (build order Multi-Agent Mathe...

Numerical Computation Utility: High-performance fallback via NumPy/SciPy when symbolic solutions are impossible (Abel-Ruffini theorem preventing closed-form solutions for quintics). (Phase 2 Coding Strategy for Ve...

Every new Supervisor and Specialist must register capabilities with the Directory Facilitator using service name plus key properties (e.g., (service: math.calculus.integration) with (type:symbolic), (algorithm:risch)). The Orchestrator queries DF in real-time, automatically discovering newly created agents without code modification. (Phase 2 Coding Strategy for Ve... (build order Multi-Agent Mathe...

Industry frameworks validate this hierarchical approach. LangGraph's **Supervisor Pattern** implements central orchestrators managing specialized workers—the most common production architecture. CrewAI's role-based crews demonstrate that well-designed task allocation elevates even simple agents, while poor task design fails even perfect agents. The "80/20 Rule" applies: 80% effort on task design, 20% on agent definition. (CrewAI)

Phase 2 Outcome: "Fragile Genius"—immense verifiable mathematical power but brittle without precondition validation or conflict resolution, necessitating Phase 3. (Phase 2 Coding Strategy for Ve...

The six-gap framework diagnoses architectural brittleness

The project documentation identifies **six foundational gaps** that plague standard "flat" multi-agent systems, providing a systematic framework for architectural hardening:

Gap 1 (Search Gap): Standard models assume the correct solution path is obvious, executing the first valid move without exploring alternatives. This causes dead ends, wasted computation, and suboptimal solutions.

(The Architectural Imperative- ...)

Gap 2 (Memory Gap): Agents are isolated and stateless—if Agent A derives a lemma, Agent B remains unaware. This creates redundant computation, inconsistency, and systemic "amnesia." (foundational gaps)

(The Architectural Imperative- ...)

Gap 3 (Assumption Gap): Systems attempt to solve problems without validating constraints, leading to confident but mathematically invalid outputs (The Architectural Imperative- ...) (e.g., finding real roots for $x^2 + 1 = 0$). (foundational gaps)

Gap 4 (Conflict Gap): When agents disagree (Symbolic claims "unsolvable" while Numerical finds approximation), systems crash or pick randomly, producing unreliable outputs. (foundational gaps +2)

Gap 5 (Reliability Gap): Generic "ERROR" messages provide no diagnosis or recovery mechanisms, creating black-box behavior and catastrophic failures. (foundational gaps) (The Architectural Imperative- ...)

Gap 6 (Evolution Gap): Static systems solve the 1000th cubic equation with the same trial-and-error as the first, showing no improvement over time. (foundational gaps) (The Architectural Imperative- ...)

These gaps align with industry-recognized challenges. PwC's multi-agent validation framework emphasizes pre-deployment testing to identify failure modes and post-deployment monitoring for continuous validation. The CoALA (Cognitive Architectures for Language Agents) paper from Princeton taxonomizes memory types (Semantic for facts, Episodic for experiences, Procedural for how-to) that directly map to Gap 2 solutions.

Phase 3 installs the cognitive immune system addressing pre-execution validation

Phase 3 shifts the control loop from (Plan → Solve) to (Analyze → Hypothesize → Validate → Solve), installing three critical teams that create resilience, context, and foresight. (Phase 3 installs the "cognitive i...)

(build order Multi-Agent Mathe...)

Precondition Validation Team ("Anesthesiologist") addresses Gap 3 through a "Pre-Flight Checklist" protocol. The Domain Checker Agent audits problems against solver capabilities—if input is a Diophantine equation but only real arithmetic solvers exist, execution halts immediately with (DOMAIN_MISMATCH).

(Phase 3 installs the "cognitive i...) The Assumption Validator scans OMDoc structures for constraint violations: (ln(x)) requires $x > 0$, (sqrt(x)) in real domain requires $x \geq 0$, (1/x) requires $x \neq 0$. The Edge Case Detector acts as a proactive "saboteur" asking "What breaks if $x=0$?" or calculating (det(A)) before matrix inversion. The Constraint Propagator injects validated constraints into every downstream sub-task payload.

(Phase 3 is designated as the Me...)

(build order Multi-Agent Mathe...)

Knowledge Management Team ("Librarians") addresses Gap 2 by transforming the stateless processor into a learning engine. The Context Extractor filters signal from noise, building token-efficient context packets. The Memory Indexer archives results to Vector Database on supervisor confirmation with embedding plus metadata (type, timestamp, source agent, confidence). The Retrieval Specialist implements RAG integration, querying Mathlib, Loogle, and internal history before solver attempts—achieving **15-20% accuracy improvement** via cached theorem retrieval. (Phase 3 is designated as the Me...)

(build order Multi-Agent Mathe...)

Hypothesis Generation Team ("Scouts") addresses Gap 1 through Tree-of-Thoughts reasoning. The Hypothesis Generator proposes strategies (Proof by Induction, Contradiction, Direct Algebraic Manipulation) without solving—ideation only. The Path Evaluator scores feasibility based on problem characteristics (recursive structure → Induction: 90%, Contradiction: 20%). The Backtracking Manager saves Blackboard snapshots before solving, restoring state and triggering Plan B upon failure. (Phase 3 is designated as the Me...)

(build order Multi-Agent Mathe...)

Industry implementations validate these patterns. LangChain/LangGraph memory architecture uses checkpoints for short-term memory and vector databases with namespaced storage for long-term memory. Mem0 provides intelligent filtering with priority scoring, dynamic forgetting with relevance decay, and hierarchical storage. (Mem0) The **ReAct Pattern** (Reasoning + Acting) implements iterative loops alternating reasoning and tool use, creating audit trails that prevent premature conclusions—(MachineLearningMastery) directly analogous to the Hypothesis Generation Team.

Phase 3 Orchestrator Protocol Updates:

- "Pre-Flight" Check: Orchestrator CANNOT delegate until `Precondition_Validation_Team.status == "VALID"`
- "Look-Before-You-Leap": Query Knowledge Management before solving; high-confidence retrieval skips solving
- "Scouting" Protocol: For high-complexity problems, invoke Hypothesis Generator FIRST

(Phase 3 is designated as the Me...)

Phase 4 transforms the static hierarchy into a dynamic self-correcting organism

Phase 4 implements the **Self-Correction Engine** with dynamic governance, addressing the three remaining gaps through specialized teams that enable evidence-based adjudication, graceful failure handling, and continuous improvement. (Phase 4 transforms this collecti...)

Conflict Resolution Team ("Supreme Court") addresses Gap 4 through the **FMAD (Feedback-based Multi-Agent Debate)** protocol. The Debate Moderator manages the "courtroom" without solving math—when Blackboard detects conflicting results for the same subtask-id, workflow freezes and the Moderator issues (PROPOSE-ARGUMENT) commands forcing structured justification ("I used Risch Algorithm" vs. "I used Newton-Raphson"). (Phase 4 transforms this collecti...) The Evidence Weigher evaluates justification quality through a **hard-coded hierarchy of mathematical truth**: Formal Proof (highest) → Symbolic Derivation → Numerical Approximation → Heuristic Guess (lowest). If Agent A provides formal proof and Agent B provides heuristic, Agent A wins automatically regardless of confidence. The Consensus Builder implements weighted voting when no formal proof exists—Domain Supervisors receive 2x weight on domain problems.

(Phase 4 transforms this collecti...)

(build order Multi-Agent Mathe...)

Research validates multi-agent debate: Du et al.'s ICML 2024 paper demonstrates that multiple LLM instances proposing and debating responses significantly enhances mathematical and strategic reasoning while reducing hallucinations. Accuracy improves with 3+ agents and 2+ debate rounds. (Composable-models)

Failure Analysis Team ("Trauma Surgeons") addresses Gap 5 with graceful incompleteness handling. The Error Classifier implements a three-category taxonomy: Computational Errors (timeout, overflow, memory exhaustion) → Request more resources and retry; Logical Errors (invalid inference, "Step 3 does not follow Step 2") → Trigger Thinker refinement loop; Domain Errors (method inapplicable) → Trigger alternative strategy. The Root Cause Analyzer traces errors to specific steps and agents, preventing blame misdirection. The Alternative Path Generator connects to Hypothesis Generator for "Plan B" construction, enabling self-healing without user intervention. (build order Multi-Agent Mathe...)

Industry error handling patterns emphasize **graceful degradation** with explicit fallback paths, **strategic redundancy** through algorithmic diversity, and **checkpoint mechanisms** for recovery. Platforms like Temporal.io provide durable workflow state persistence with automatic retry, while Elixir's Actor Model implements "let it crash" philosophy with supervision trees for automatic process restart.

Meta-Learning Team ("Optimizer") addresses Gap 6 by installing memory of process, not just facts. The Performance Monitor logs traces of every successful solution (problem_type, agent_sequence, time_taken, verification_status, resource_usage). The Agent Selector Optimizer implements **AutoMaAS (Automated Multi-Agent System)** logic, analyzing logs to identify patterns ("For optimization problems: Geometric_Agent fails 80%, Gradient_Descent_Agent succeeds 85%") and generating updated routing tables. The Adaptive Dispatcher connects optimizer to orchestrator with dynamic scaling: Simple queries deploy "Skeleton Crew" (2-3 agents) while complex queries deploy "Full Debate Team" (10+ agents), achieving **10-15% cost-per-token reduction**. (Phase 4 transforms this collecti... (build order Multi-Agent Mathe...)

Phase 4 Protocol Updates:

- "Appellate" Protocol: Before accepting any result, check for CONFLICT_FLAG and delegate to Debate Moderator
- "Post-Mortem" Protocol: On session end, Meta_Learning_Team runs batch process to analyze traces, update routing weights, and persist improvements (Phase 4 transforms this collecti...)

Phase 5 distills massive deliberative intelligence into fast intuitive responses

Phase 5 shifts focus to efficiency (latency, cost, scale) through **Hybrid Deployment**—utilizing a fast "distilled" single agent for standard queries while reserving the massive multi-agent swarm for complex reasoning. The architectural paradigm transforms System 2 intelligence (slow, deliberative 40-50 agent hierarchy) into System 1 efficiency (fast, intuitive single model) without losing deep complexity handling. (Phase 5 is the Production Opti...)

"Thought Trace" Harvest implements the Provenance Logger—an upgraded Performance Monitor capturing full causal chains: User Query → Orchestrator Decomposition → Supervisor Strategy Selection → Specialist

Execution → Verification Steps → Final Output. (Phase 5 is the Production Opti...) The critical **Gold Standard Filter** commits traces to training set ONLY if they receive STATUS: VERIFIED from the Ax-Prover loop or consensus from the Debate Moderator, ensuring the Student model learns exclusively from successful reasoning paths. (Phase 5 is the Production Opti...)

The Distillation Engine trains a 7B-8B parameter Student model to predict: (1) internal routing logic of the Orchestrator, (2) algorithmic choices of Supervisors. Result metrics: **1/5th the latency and cost** of the full MAS, with a single model simulating 50-agent swarm reasoning for standard problems.

(Phase 5 shifts the focus to effici...) (Phase 5 is the Production Opti...)

The Complexity Gatekeeper ("Triage Nurse") positions as a lightweight classifier at system edge BEFORE the Orchestrator, routing standard queries (high-frequency, low-complexity like "Calculate derivative of x^2 ") to the Distilled Student Model and novel/high-stakes queries to the Full Tier-3 Multi-Agent Hierarchy. A **Confidence Fallback Mechanism** automatically escalates to full MAS if Distilled Model confidence falls below threshold or fails Ax-Prover verification, creating seamless "Fail-Over" to higher intelligence.

(Phase 5 is the Production Opti...)

The Evolutionary Flywheel implements active learning: when Distilled Model fails AND escalates to MAS AND MAS successfully resolves, the solution is tagged as "High-Priority Training Data" and queued for retraining. The Student gradually masters hard cases, constantly raising baseline intelligence.

(Phase 5 is the Production Opti...)

Operational Hardening adds a User Simulator Agent ("Stress Tester") bombarding the system with edge cases, mathematically ambiguous queries, and prompt injection attacks. (Phase 5 shifts the focus to effici...) An Identity Manager (IAM Agent) assigns unique identities enforcing Least-Privilege Access—Graph Theory Agent cannot access memory logs of Financial Optimization Agent. (Phase 5 shifts the focus to effici...)

Industry validation comes from AlphaProof's approach: Test-Time Reinforcement Learning (TTRL) generates millions of problem variants during inference, enabling continuous self-improvement. The LangGraph benchmarks show it achieves **lowest latency across data analysis tasks**, (AIMultiple) validating the architectural approach of fast-path routing with fallback to deliberative systems.

Phase 5 Deliverable: "The Apex System"—fast single model handles 90% of interactions, instantly unfolds into 50-agent swarm for complex problems, autonomously updates "instincts" based on MAS successes.

Phase 6 engineers the capacity for novel mathematical discovery

Phase 6 represents the transition from Problem-Solving Engine to Mathematical Discovery Engine. While Phases 1-5 constructed systems solving known problems, Phase 6 must handle infinite search spaces, undecidability, and heuristic algorithm generation—moving beyond standard BDI execution loops.

(Phase Six: Discovery and Expa...)

Conjecture Generation Team ("Theorist") evolves the system from reactive problem solving to proactive knowledge synthesis. (Phase Six: Discovery and Expa...) The Synthetic Data Generator ("The Dreamer") implements

the AlphaGeometry paradigm—continuously generating random geometric/algebraic premises, deriving conclusions using Phase 2 symbolic engines, producing millions of "synthetic theorems" as logically true statements for training data. The Pattern Recognizer ("The Filter") sifts through millions of synthetic theorems, using heuristics to filter trivial tautologies and identify interesting non-trivial relationships, elevating discoveries to "Candidate Conjectures." The Conjecture Formalizer translates raw relationships into rigorous Lean or Isabelle statements. (Phase Six: Discovery and Expa...)

Deep Search Team ("The Explorer") navigates infinite search spaces where standard Tree-of-Thoughts is insufficient. The Policy Network Agent ("The Tactician") implements AlphaProof Prover architecture—generating massive search trees of potential proof tactics probabilistically, suggesting next moves based on synthetic data patterns. The Critic Network Agent ("The Evaluator") estimates success probability for specific branches WITHOUT full expansion, enabling early pruning of dead ends. The Search Tree Manager implements **Product-Node search architecture**, tracking thousands of parallel proof attempts, prioritizing promising branches, and suspending low-value exploration. (Phase Six: Discovery and Expa...)

Algorithm Discovery Unit (FunSearch Pattern) finds algorithms rather than answers. The Code Evolutionary Proposer generates Python/Julia/Lean snippets representing heuristics using evolutionary logic—take best code from previous generation, mutate for optimization. The Sandbox Evaluator executes proposed code against rigorous test sets, returning scalar scores (execution speed, compression ratio) and filtering incorrect code immediately. The Heuristic Distiller analyzes successful evolutionary code, extracting underlying mathematical principles and converting "black box" success into human-readable heuristics. (Phase Six: Discovery and Expa...)

Undecidability Navigator ("Boundary Watcher") protects against hard mathematical limits. The Decidability Checker inspects logical structure before Deep Search commitment, classifying theories as Decidable (Presburger Arithmetic, Real Closed Fields → use complete decision procedures) or Undecidable (Peano Arithmetic, Diophantine Equations → switch to Heuristic Search Mode). The Interactive Guidance Liaison implements Human-in-the-Loop interface—when Deep Search hits theoretical walls, it generates Proof State Summaries and requests specific human guidance. (Phase Six: Discovery and Expa...)

Formal Knowledge Integration Team ("The Archivist") ensures discoveries integrate into the system. The Auto-Formalization Pipeline converts outputs into strict OMDoc/OpenMath entries. The Vector Database Updater re-indexes system memory with new discoveries, "teaching" the entire Phase 2 workforce new mathematics Phase 6 discovered—creating compounding intelligence. (Phase Six: Discovery and Expa...)

Industry frameworks validate and extend the architectural approach

Comparing project strategies against industry best practices reveals strong alignment with additional optimization opportunities:

Project Strategy	Industry Validation	Enhancement Opportunity
3-Tier Hierarchy	LangGraph Supervisor Pattern (most common production architecture)	Consider swarm pattern for adaptive multi-turn interactions
FIPA-ACL Communication	FIPA standards remain foundational; ACP/A2A emerging for interoperability	Implement MCP for tool integration, ACP for cross-platform agent discovery
Ax-Prover Loop	AlphaProof (IMO silver), AlphaGeometry (83% geometry problems)	Add Test-Time Reinforcement Learning for continuous improvement
Multi-Agent Debate	Du et al. research: 3+ agents, 2+ rounds optimal	Tune debate rounds and adversarial stance levels
AutoMaAS Dynamic Selection	Validated 10-15% improvement over static selection	Consider ensemble methods for routing decisions
Vector Database Memory	Industry standard (Pinecone, Milvus, ChromaDB)	Add intelligent filtering with relevance decay and namespace organization

Framework Recommendations:

- **LangGraph** for production systems requiring control and lowest latency
 - **CrewAI** for role-based collaboration with simpler configuration (`GitHub`)
 - **Lean 4** as verification backend (binary verdict: mathematically correct or fails)
 - **LangSmith** for end-to-end observability, tracing, and debugging
-

Actionable coding recommendations for implementation teams

Phase 0 Implementation Priorities:

1. Implement AMS with 90% memory threshold enforcement and agent weight swapping
2. Deploy DF with capability registration schema (`service`), (`type`), (`algorithm`) properties)
3. Build ACC with message queuing for inactive agents and Conversation ID tracking
4. Initialize Vector Database in persistent mode with efficient retrieval indexing

Phase 1 Implementation Priorities:

1. Create Syntax Parser with OMDoc expression tree output
2. Implement HTN decomposition engine in Orchestrator with delegation-only constraint
3. Wrap SymPy as Pilot Solver with standard agent interface

4. Build two-layer Verification Core: rule-based Logic Checker + Lean 4 Formal Verifier

Phase 2 Implementation Priorities:

1. Implement Domain Supervisors with routing decision logic (symbolic vs. numerical)
2. Build Specialists wrapping specific algorithms (GMP, Gröbner, Risch, SVD)
3. Create DF registration automation for new agent capability discovery
4. Test dual-engine Integration Specialist with fallback path

Phase 3 Implementation Priorities:

1. Implement Domain Checker with decidable fragment database
2. Build Assumption Validator with constraint propagation
3. Create RAG-enabled Retrieval Specialist with Mathlib/Loogle integration
4. Implement Backtracking Manager with Blackboard state snapshots

Phase 4 Implementation Priorities:

1. Implement FMAD protocol with PROPOSE-ARGUMENT commands
2. Build Evidence Weigher with hard-coded mathematical truth hierarchy
3. Create Error Classifier with three-category taxonomy and remediation routing
4. Implement Performance Monitor logging and AutoMaAS routing table updates

Phase 5 Implementation Priorities:

1. Build Provenance Logger with Gold Standard filtering
2. Train 7B-8B Student model on validated thought traces
3. Implement Complexity Gatekeeper with confidence-based escalation
4. Create User Simulator for stress testing and IAM for access control

Phase 6 Implementation Priorities:

1. Implement Synthetic Data Generator producing premise-conclusion pairs
2. Build Pattern Recognizer with triviality filtering heuristics
3. Create Policy/Critic Network pair for Product-Node search
4. Implement Sandbox Evaluator with secure code execution
5. Build Human-in-the-Loop interface for undecidability handling

Scalability and production considerations

Resource Management Strategy:

- Stateful Serialization Protocol with one active cognitive model
- Tiered communication: Events (local) → RMI (inter-container) → IIOP (inter-platform)
- Vector Database namespace organization: `domain/agent-type/specific-instance`

Error Handling Architecture:

- Implement exponential backoff with max retries for transient failures
- Circuit breaker pattern for agent dependencies ([Microsoft Learn](#))
- Graceful degradation maintaining critical functions under reduced capability
- Checkpoint mechanisms enabling recovery from any interruption point

Monitoring and Observability:

- LangSmith tracing for all agent operations with failure filtering
- Token consumption and latency tracking per agent
- Self-Reflection Rate (SRR) metrics for meta-cognitive evaluation
- Hallucination Rate targeting below 5%

Testing Strategy:

- Smoke tests for basic agent functionality
- Integration tests validating agent coordination and communication
- Scalability stress tests under load
- Adversarial testing for edge cases and prompt injection vulnerabilities
- Continuous evaluation against mathematical benchmarks (GSM8K, MATH)

The evolutionary trajectory from Phase 0 infrastructure through Phase 6 discovery capabilities represents not incremental improvement but architectural transformation. Each phase addresses specific gaps while preparing foundations for subsequent capabilities. The system at Day N+1 is guaranteed to outperform the system at Day N through embedded meta-learning—representing the ultimate goal of autonomous mathematical discovery.